

THE INTERVIEW

API Reference Documentation

Complete Endpoint Specifications for All Microservices

Gateway Base URL: <http://localhost:5190>

Auth Scheme: Bearer JWT · Admin Role · Internal Key

Services: Identity · Interview · Assessment · Subscription

Common Rules

Most endpoints require these HTTP headers:

HTTP

```
Authorization: Bearer <jwt-token>
Content-Type: application/json
```

Anonymous endpoints are marked accordingly. Admin endpoints require a JWT with role Admin.

All successful responses follow this envelope:

JSON

```
{
  "success": true,
  "message": "Request completed successfully.",
  "data": {}
}
```

All error responses follow this envelope:

JSON

```
{
  "success": false,
  "message": "Validation failed.",
  "details": "The Email field is required."
}
```

Service Ports

Service	Direct URL
API Gateway	http://localhost:5190
Identity Service	http://localhost:5005
Interview Service	http://localhost:5002
Assessment Service	http://localhost:5003
Subscription Service	http://localhost:5004

Quick Endpoint Index

Method	Path	Auth
POST	/api/auth/register	Anonymous
POST	/api/auth/login	Anonymous
POST	/api/auth/forgot-password/request-otp	Anonymous
POST	/api/auth/forgot-password/reset	Anonymous
GET	/api/auth/me	User
POST	/api/auth/refresh-claims	User
PUT	/api/auth/me	User
GET	/api/auth/admin/users	Admin
GET	/api/auth/admin/users/{id}	Admin
PUT	/api/auth/admin/users/{id}/role	Admin
PUT	/api/auth/admin/users/{id}/premium	Admin
PUT	/api/auth/admin/users/{id}/deactivate	Admin
PUT	/api/auth/admin/users/{id}/reactivate	Admin
PUT	/api/auth/internal/users/{id}/premium	Internal Key
GET	/api/auth/notifications	User
PUT	/api/auth/notifications/read	User
POST	/api/interviews/start	User
POST	/api/interviews/{id}/begin	User
POST	/api/interviews/warm-up	User
POST	/api/interviews/{id}/fetch-more	User
POST	/api/interviews/submit	User
GET	/api/interviews	User
GET	/api/interviews/{id}	User
GET	/api/interviews/{id}/result	User
GET	/api/interviews/admin/all	Admin
POST	/api/assessments/start	User
GET	/api/assessments/{id}/next-batch	User
POST	/api/assessments/warm-up	User
POST	/api/assessments/submit	User
GET	/api/assessments	User
GET	/api/assessments/{id}/result	User
GET	/api/assessments/admin/all	Admin
GET	/api/assessments/questions	Admin
POST	/api/assessments/questions	Admin
PUT	/api/assessments/questions/{id}	Admin
DELETE	/api/assessments/questions/{id}	Admin
POST	/api/subscriptions/subscribe	User
POST	/api/subscriptions/confirm	User
POST	/api/subscriptions/webhook/stripe	Anonymous / Stripe
POST	/api/subscriptions/cancel	User
GET	/api/subscriptions/my	User

Method	Path	Auth
GET	/api/subscriptions/my/payments	User
GET	/api/subscriptions/all	Admin
GET	/api/subscriptions/payments	Admin

1. Identity Service

Base Path: /api/auth

1.1 Register User

POST /api/auth/register

Auth Anonymous

Creates a new candidate account, generates a JWT token, publishes a user registration event, and sends a welcome email notification.

Request Body:

JSON

```
{
  "fullName": "Shobhit Sharma",
  "email": "shobhit@example.com",
  "password": "Password123"
}
```

Response:

JSON

```
{
  "success": true,
  "message": "Registration successful",
  "data": {
    "userId": 1,
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.demo.token",
    "message": "Registration successful"
  }
}
```

1.2 Login

POST /api/auth/login

Auth Anonymous

Authenticates an existing user with email and password, then returns a JWT token used for protected APIs.

Request Body:

JSON

```
{
  "email": "shobhit@example.com",
  "password": "Password123"
}
```

```
}
```

Response:

JSON

```
{
  "success": true,
  "message": "Login successful",
  "data": {
    "userId": 1,
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.demo.token",
    "message": "Login successful"
  }
}
```

1.3 Request Forgot Password OTP

POST /api/auth/forgot-password/request-otp

Auth Anonymous

Starts the password reset flow by generating a 6-digit OTP and sending it to the user's registered email address when the account exists.

Request Body:

JSON

```
{
  "email": "shobhit@example.com"
}
```

Response:

JSON

```
{
  "success": true,
  "message": "If the email exists, an OTP has been sent.",
  "data": null
}
```

1.4 Reset Password With OTP

POST /api/auth/forgot-password/reset

Auth Anonymous

Validates the password reset OTP and replaces the user's password with the new password.

Request Body:

JSON

```
{
  "email": "shobhit@example.com",
  "otp": "123456",
  "newPassword": "NewPassword123"
}
```

Response:

JSON

```
{
```


Request Body:

JSON

```
{
  "fullName": "Shobhit S."
}
```

Response:

JSON

```
{
  "success": true,
  "message": "Profile updated successfully.",
  "data": {
    "userId": 1,
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.updated.token",
    "message": "Token refreshed successfully"
  }
}
```

1.8 Get All Users

GET /api/auth/admin/users**Auth** Admin

Returns all registered users so administrators can review accounts, roles, premium state, and activation status.

Request Body: None

Response:

JSON

```
{
  "success": true,
  "message": "Request completed successfully.",
  "data": [
    {
      "id": 1,
      "fullName": "Shobhit Sharma",
      "email": "shobhit@example.com",
      "role": "Candidate",
      "isPremium": true,
      "isActive": true,
      "createdAt": "2026-05-02T10:30:00Z"
    }
  ]
}
```

1.9 Get User By ID

GET /api/auth/admin/users/{id}**Auth** Admin

Returns one user account by id for administrative review or account management.

Request Body: None

Response:

JSON

```
{
  "success": true,
  "message": "Request completed successfully.",
  "data": {
    "id": 1,
    "fullName": "Shobhit Sharma",
    "email": "shobhit@example.com",
    "role": "Candidate",
    "isPremium": true,
    "isActive": true,
    "createdAt": "2026-05-02T10:30:00Z"
  }
}
```

1.10 Update User Role

PUT /api/auth/admin/users/{id}/role

Auth Admin

Updates a user's role — for example, promoting a Candidate account to Admin.

Request Body:

JSON

```
"Admin"
```

Response:

JSON

```
{
  "success": true,
  "message": "Role updated.",
  "data": null
}
```

1.11 Update User Premium Status

PUT /api/auth/admin/users/{id}/premium

Auth Admin

Manually updates a user's premium flag from the admin panel.

Request Body:

JSON

```
true
```

Response:

JSON

```
{
  "success": true,
  "message": "Premium status updated.",
  "data": null
}
```

1.12 Deactivate User

PUT /api/auth/admin/users/{id}/deactivate

Auth *Admin*

Deactivates a user account so the user can no longer use the platform normally.

Request Body: None

Response:

JSON

```
{
  "success": true,
  "message": "User deactivated.",
  "data": null
}
```

1.13 Reactivate User

PUT /api/auth/admin/users/{id}/reactivate**Auth** *Admin*

Reactivates a previously deactivated user account.

Request Body: None

Response:

JSON

```
{
  "success": true,
  "message": "User reactivated.",
  "data": null
}
```

1.14 Internal Update Premium

PUT /api/auth/internal/users/{id}/premium**Auth** *Anonymous*

Internal service-to-service endpoint used to update premium status after subscription lifecycle events. The API Gateway intentionally blocks the /api/auth/internal/* route — call this directly on the Identity Service (<http://localhost:5005>) from trusted backend code.

Request Body:

JSON

```
{
  "isPremium": true
}
```

Response:

JSON

```
{
  "success": true,
  "message": "Premium status updated via internal call.",
  "data": null
}
```

1.15 Get Notifications

GET /api/auth/notifications

Auth Bearer Token

Returns the latest 50 notifications for the current user, ordered newest first.

Request Body: None

Response:

JSON

```
{
  "success": true,
  "message": "Request completed successfully.",
  "data": [
    {
      "id": 10,
      "userId": 1,
      "title": "Assessment completed",
      "message": "Your C# assessment result is ready.",
      "actionUrl": "/assessments/12/result",
      "isRead": false,
      "createdAt": "2026-05-02T11:00:00Z"
    }
  ]
}
```

1.16 Mark Notifications As Read

PUT /api/auth/notifications/read

Auth Bearer Token

Marks all unread notifications for the current user as read.

Request Body: None

Response:

JSON

```
{
  "success": true,
  "message": "Notifications marked as read",
  "data": null
}
```

2. Interview Service

Base Path: /api/interviews

2.1 Start Interview

POST /api/interviews/start

Auth Bearer Token

Creates a new interview session in Pending state for the selected title and domain.

Request Body:

JSON

```
{
  "title": "Senior .NET Mock Interview",
  "domain": ".NET Backend"
}
```

Response:

JSON

```
{
  "success": true,
  "message": "Interview created successfully.",
  "data": {
    "id": 21,
    "userId": 1,
    "title": "Senior .NET Mock Interview",
    "domain": ".NET Backend",
    "type": "Premium",
    "status": "Pending",
    "createdAt": "2026-05-02T11:10:00Z",
    "startedAt": null,
    "completedAt": null
  }
}
```

2.2 Begin Interview

POST /api/interviews/{id}/begin

Auth Bearer Token

Starts a pending interview, transitions it to In-Progress, and returns the ordered list of questions for the session.

Request Body: None

Response:

JSON

```
{
  "success": true,
  "message": "Request completed successfully.",
  "data": {
    "interviewId": 21,
    "totalExpected": 5,
    "message": "All 5 questions loaded.",
    "questions": [
      {
        "id": 1,
        "text": "Explain dependency injection in ASP.NET Core.",
        "questionType": "Subjective",
        "orderIndex": 1
      }
    ]
  }
}
```

2.3 Warm Up Interview Question Cache

POST /api/interviews/warm-up**Auth** Bearer Token

Pre-generates and caches interview questions for a domain so the actual interview can begin faster.

Request Body:

JSON

```
{
  "domain": ".NET Backend",
  "targetCount": 5
}
```

Response:

JSON

```
{
  "success": true,
  "message": "Warm-up triggered.",
  "data": { "count": 5 }
}
```

2.4 Fetch More Interview Questions

POST /api/interviews/{id}/fetch-more**Auth** Bearer Token

Generates or retrieves additional questions for an in-progress interview when more questions are needed.

Request Body: None

Response: Same structure as Begin Interview response.

2.5 Submit Interview

POST /api/interviews/submit**Auth** Bearer Token

Submits the user's answers, evaluates them via AI, stores the result, and sends completion email notifications. Premium users receive a detailed breakdown.

Request Body:

JSON

```
{
  "interviewId": 21,
  "answers": [
    {
      "questionId": 1,
      "answerText": "Dependency injection provides dependencies through constructors or containers."
    }
  ]
}
```

Response (Standard User):

JSON

```
{
  "success": true,
  "message": "Request completed successfully.",
  "data": {
    "totalScore": 8,
    "maxScore": 10,
    "percentage": 80,
    "grade": "A",
    "wrongQuestionIds": [],
    "feedback": "Good answer. Add a concrete production example."
  }
}
```

Response (Premium User — includes breakdown, strengths, weak areas):

JSON

```
{
  "success": true,
  "data": {
    "totalScore": 8,
    "maxScore": 10,
    "percentage": 80,
    "grade": "A",
    "breakdown": [
      {
        "text": "Explain dependency injection in ASP.NET Core.",
        "subtopic": "Dependency Injection",
        "yourAnswer": "...",
        "correctAnswer": "...",
        "isCorrect": true,
        "score": 8
      }
    ],
    "strengths": ["Dependency Injection"],
    "weakAreas": [],
    "suggestions": [],
    "feedback": "Good answer. Add a concrete production example."
  }
}
```

2.6 Get My Interviews

GET /api/interviews

Auth Bearer Token

Returns all interview sessions created by the current authenticated user.

Request Body: None

Response:

JSON

```
{
  "success": true,
  "data": [
    {
      "id": 21,
      "title": "Senior .NET Mock Interview",
      "domain": ".NET Backend",
      "type": "Premium",
      "status": "Completed",
      "createdAt": "2026-05-02T11:10:00Z",
      "startedAt": "2026-05-02T11:11:00Z",

```

```
    "completedAt": "2026-05-02T11:25:00Z"  
  }  
]  
}
```

2.7 Get Interview Details

GET /api/interviews/{id}

Auth Bearer Token

Returns one interview owned by the current user together with its saved question list.

Request Body: None

Response: Interview object plus questions array (same shape as Begin Interview).

2.8 Get Interview Result

GET /api/interviews/{id}/result

Auth Bearer Token

Returns the calculated result for a completed interview. Premium users receive a richer review breakdown.

Request Body: None

Response: Same structure as Submit Interview response for the user's tier.

2.9 Get All Interviews (Admin)

GET /api/interviews/admin/all

Auth Admin

Returns every interview session in the system for admin monitoring and reporting.

Request Body: None

Response: Array of all interview objects across all users.

3. Assessment Service

Base Path: /api/assessments

3.1 Start Assessment

POST /api/assessments/start

Auth Bearer Token

Creates a new timed MCQ assessment for the selected domain, prepares the first question batch, and returns timing details.

Request Body:

JSON

```
{
  "domain": "C#",
  "questionCount": 10,
  "difficulty": "Medium"
}
```

Response:

JSON

```
{
  "success": true,
  "data": {
    "assessmentId": 12,
    "timeLimitMinutes": 10,
    "expiresAt": "2026-05-02T11:40:00Z",
    "questions": [
      {
        "id": 101,
        "text": "Which DI lifetime is normally used for DbContext in ASP.NET Core?",
        "optionA": "Singleton",
        "optionB": "Transient",
        "optionC": "Scoped",
        "optionD": "Static",
        "orderIndex": 1
      }
    ],
    "totalExpected": 10,
    "hasMore": true
  }
}
```

3.2 Get Next Batch

GET /api/assessments/{id}/next-batch?currentCount=5&batchSize=5

Auth Bearer Token

Returns the next batch of MCQ questions for a running assessment using currentCount and batchSize for lazy loading.

Request Body: None

Response:

JSON

```
{
  "success": true,
  "data": [
    {
      "id": 106,
      "text": "What problem does the Circuit Breaker pattern solve?",
      "optionA": "Authentication",
      "optionB": "Partial failures in downstream services",
      "optionC": "SQL indexing",
      "optionD": "CSS caching",
      "orderIndex": 6
    }
  ]
}
```

3.3 Warm Up Assessment Question Cache

POST /api/assessments/warm-up

Auth Bearer Token

Pre-generates and caches MCQ questions for a domain and difficulty before the user starts the assessment.

Request Body:

JSON

```
{
  "domain": "C#",
  "difficulty": "Medium",
  "targetCount": 5
}
```

Response:

JSON

```
{
  "success": true,
  "message": "Warm-up complete.",
  "data": { "cachedCount": 5 }
}
```

3.4 Submit Assessment

POST /api/assessments/submit

Auth Bearer Token

Submits selected MCQ answers, calculates score, grade, and percentage, stores the result, and publishes assessment completion notifications. Premium users receive answer review and weak-area analysis.

Request Body:

JSON

```
{
  "assessmentId": 12,
  "answers": [
    { "questionId": 101, "selectedOption": "C" }
  ],
  "totalExpected": 10
}
```

Response (Standard User):

JSON

```
{
  "success": true,
  "data": {
    "score": 1, "maxScore": 10, "percentage": 10,
    "grade": "F", "wrongQuestionIds": []
  }
}
```

Response (Premium User — includes answer review and weak areas):

JSON

```
{
```



```
{
  "success": true,
  "data": {
    "score": 8, "maxScore": 10, "percentage": 80, "grade": "A",
    "wrongQuestionIds": [104, 109],
    "answerReview": [
      {
        "text": "Which DI lifetime is normally used for DbContext in ASP.NET Core?",
        "subtopic": "Dependency Injection",
        "selectedOption": "C", "correctOption": "C", "isCorrect": true
      }
    ],
    "weakAreas": ["Concurrency"],
    "weakAreasSummary": "Weak areas identified from incorrect answers."
  }
}
```

3.5 Get My Assessments

GET /api/assessments

Auth Bearer Token

Returns the current user's completed assessment result history.

Request Body: None

Response:

JSON

```
{
  "success": true,
  "data": [
    {
      "id": 7, "assessmentId": 12, "userId": 1,
      "domain": "C#", "score": 8, "maxScore": 10,
      "percentage": 80, "grade": "A",
      "isPremiumResult": true, "createdAt": "2026-05-02T11:35:00Z"
    }
  ]
}
```

3.6 Get Assessment Result

GET /api/assessments/{id}/result

Auth Bearer Token

Returns the result for one assessment owned by the current user. Premium users receive answer review and weak-area analysis.

Request Body: None

Response: Same structure as Premium Submit Assessment response.

3.7 Get All Assessments (Admin)

GET /api/assessments/admin/all

Auth Admin

Returns all assessment results in the system for admin dashboards and reporting.

Request Body: None

Response: Array of all assessment result objects across all users.

3.8 Get All Questions

GET /api/assessments/questions

Auth Admin

Returns the full MCQ question bank used by assessments.

Request Body: None

Response:

JSON

```
{
  "success": true,
  "data": [
    {
      "id": 101, "domain": "C#",
      "text": "Which DI lifetime is normally used for DbContext in ASP.NET Core?",
      "optionA": "Singleton", "optionB": "Transient",
      "optionC": "Scoped", "optionD": "Static",
      "correctOption": "C", "subtopic": "Dependency Injection",
      "marks": 1, "orderIndex": 1
    }
  ]
}
```

3.9 Add Question

POST /api/assessments/questions

Auth Admin

Adds a new MCQ question to the assessment question bank.

Request Body:

JSON

```
{
  "domain": "C#",
  "text": "Which collection supports thread-safe producer-consumer scenarios?",
  "optionA": "List<T>", "optionB": "Dictionary<TKey,TValue>",
  "optionC": "BlockingCollection<T>", "optionD": "ArrayList",
  "correctOption": "C", "subtopic": "Concurrency"
}
```

Response:

JSON

```
{
  "success": true,
  "message": "Question added successfully.",
  "data": { "id": 130, "domain": "C#", "marks": 1, "orderIndex": 0 }
}
```

3.10 Update Question

PUT /api/assessments/questions/{id}

Auth Admin

Updates an existing MCQ question including its domain, text, options, correct answer, and subtopic.

Request Body: Same structure as Add Question.

Response:

JSON

```
{
  "success": true,
  "message": "Question updated successfully.",
  "data": { "id": 130, "domain": "C#", "marks": 1, "orderIndex": 0 }
}
```

3.11 Delete Question

DELETE /api/assessments/questions/{id}

Auth *Admin*

Deletes an MCQ question from the assessment question bank.

Request Body: None

Response:

JSON

```
{
  "success": true,
  "message": "Question deleted successfully.",
  "data": null
}
```

4. Subscription Service

Base Path: /api/subscriptions

4.1 Subscribe

POST /api/subscriptions/subscribe

Auth *Bearer Token*

Creates a pending premium subscription payment session. In Stripe mode it returns a checkout URL; in local demo mode it returns a simulated session id.

Request Body: None

Response (Stripe enabled):

JSON

```
{
  "success": true,
  "data": {
    "checkoutSessionId": "cs_test_123",
  }
}
```

```

    "checkoutUrl": "https://checkout.stripe.com/c/pay/cs_test_123",
    "publishableKey": "pk_test_123",
    "amount": 99900, "currency": "INR", "mode": "Stripe",
    "message": "Redirect the user to the Stripe checkout URL."
  }
}

```

Response (Simulated / local demo mode):

JSON

```

{
  "success": true,
  "data": {
    "checkoutSessionId": "sim_session_1_638818530000000000",
    "publishableKey": "pk_test_simulated",
    "amount": 99900, "currency": "INR", "mode": "Simulated",
    "message": "Stripe is disabled. Use the confirm endpoint for local demo mode."
  }
}

```

4.2 Confirm Payment

POST /api/subscriptions/confirm

Auth Bearer Token

Confirms a local demo payment, activates the user's premium subscription, records the payment, and publishes premium activation events via RabbitMQ.

Request Body:

JSON

```

{
  "paymentSessionId": "sim_session_1_638818530000000000",
  "paymentReferenceId": "pay_demo_123",
  "signature": "demo_signature"
}

```

Response:

JSON

```

{
  "success": true,
  "data": {
    "transactionId": "pay_demo_123", "amount": 999, "status": "Success",
    "startDate": "2026-05-02T11:45:00Z", "endDate": "2026-06-01T11:45:00Z",
    "message": "Premium activation event published. Call POST /api/auth/refresh-claims
to update JWT claims."
  }
}

```

4.3 Stripe Webhook

POST /api/subscriptions/webhook/stripe

Auth Anonymous

Receives Stripe checkout webhook events, verifies the Stripe signature, updates local payment records, and activates premium when payment succeeds.

Required Header:

HTTP

```
Stripe-Signature: t=1714640000,v1=demo_signature
```

Request Body: Raw Stripe webhook JSON

Response:

JSON

```
{
  "success": true,
  "message": "Stripe webhook processed.",
  "data": {
    "processed": true,
    "sessionId": "cs_test_123",
    "paymentIntentId": "pi_123",
    "subscriptionId": 5
  }
}
```

4.4 Cancel Subscription

POST /api/subscriptions/cancel

Auth Bearer Token

Cancels the current user's active premium subscription and starts the premium removal workflow.

Request Body: None

Response:

JSON

```
{
  "success": true,
  "message": "Subscription cancelled successfully.",
  "data": null
}
```

4.5 Get My Subscriptions

GET /api/subscriptions/my

Auth Bearer Token

Returns all subscription records for the current authenticated user.

Request Body: None

Response:

JSON

```
{
  "success": true,
  "data": [
    {
      "id": 5, "userId": 1, "plan": "Premium", "price": 999,
      "status": "Active", "sagaState": "Completed",
      "startDate": "2026-05-02T11:45:00Z",
      "endDate": "2026-06-01T11:45:00Z",
      "createdAt": "2026-05-02T11:45:00Z"
    }
  ]
}
```

```
}
```

4.6 Get My Payment History

GET /api/subscriptions/my/payments

Auth Bearer Token

Returns the payment history for the current authenticated user.

Request Body: None

Response:

JSON

```
{
  "success": true,
  "data": [
    {
      "id": 8, "subscriptionId": 5, "userId": 1,
      "amount": 999, "currency": "INR",
      "stripeSessionId": "cs_test_123",
      "stripePaymentIntentId": "pi_123",
      "status": "Success",
      "createdAt": "2026-05-02T11:45:00Z"
    }
  ]
}
```

4.7 Get All Subscriptions (Admin)

GET /api/subscriptions/all

Auth Admin

Returns every subscription record in the system for admin review.

Request Body: None

Response: Array of all subscription objects across all users.

4.8 Get All Payments (Admin)

GET /api/subscriptions/payments

Auth Admin

Returns every payment record in the system for admin finance and audit views.

Request Body: None

Response: Array of all payment objects across all users.